一、`Django`基础

二、课前准备

1、下载`Django`安装包

三、教学目标

1、掌握`Django`项目的创建、配置及启动

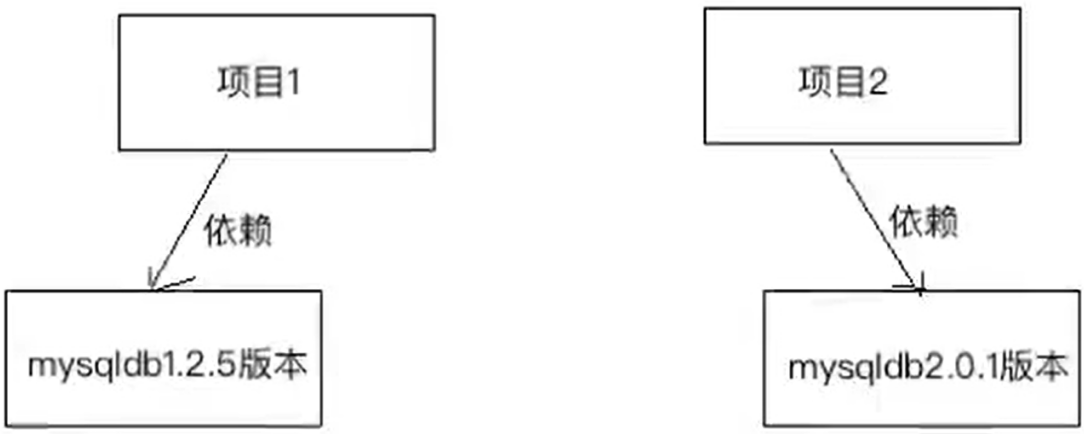
2、掌握`Django`调度器，可根据需求，同时结合`path()`或`re_path()`函数来定义`URL`调度器

3、掌握`Django`应用的创建

4、掌握`Django`模板的创建

5、能够流畅的表达出`Django`调度器的工作流程

四、创建虚拟环境



安装同一个包的不同版本，后安装的包会把原来安装的包覆盖掉。这样:如同一台机器上两个项目依赖于相同包的不同版本，则会导致一些项目运行失败

解决方案: 虚拟环境

【虚拟环境是真实 python 环境的复制版本。】

4.0.1 创建并激活 Python 虚拟环境:

虚拟环境可以隔离项目依赖，避免全局安装的包产生冲突。

步骤:

1. **打开命令提示符（CMD）或 PowerShell**
   按下 `Win + R`，输入 `cmd` 或 `powershell`，回车。
2. **创建项目文件夹并进入**

   ```
   mkdir mydjango_project
   cd mydjango_project
   ```

3. **创建虚拟环境**

   ```
   python -m venv myenv
   ```

   - 这会生成一个名为 `myenv` 的虚拟环境目录。
4. **激活虚拟环境**
   - 在 CMD 中:

     ```
     myenv\Scripts\activate
     ```

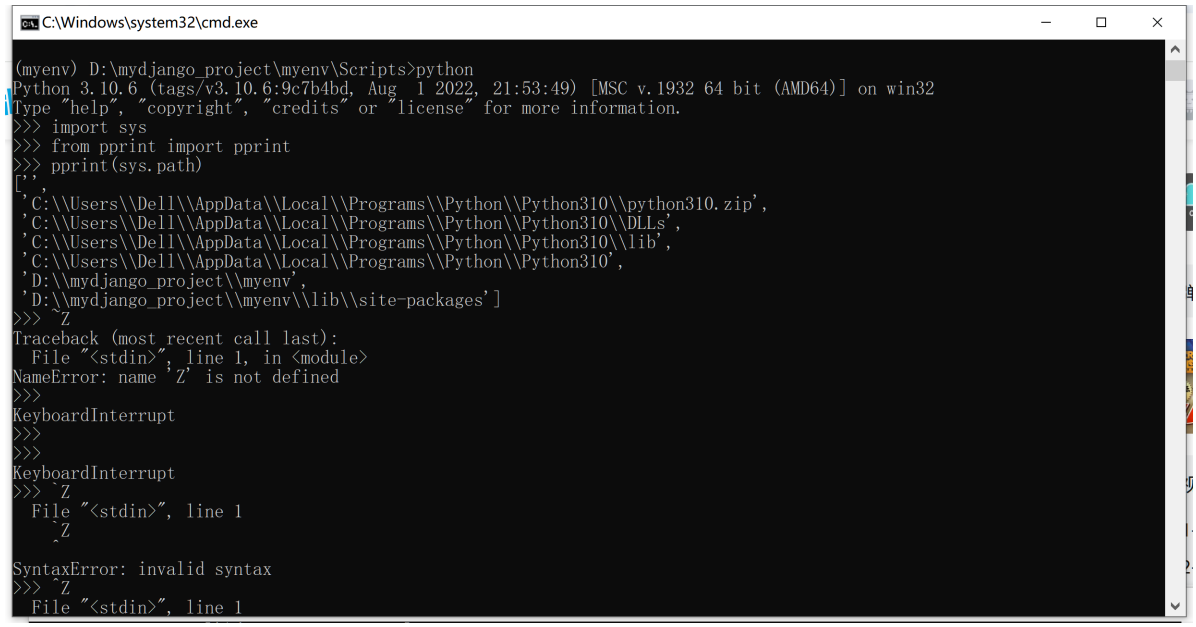
     在 PowerShell 中:

     ```
     .\myenv\Scripts\Activate.ps1
     ```

   - 激活后，命令行提示符会显示虚拟环境名称 `(myenv)`。
5. **退出虚拟环境**

```
deactivate
```

6. 查看一下环境变量



```
C:\Windows\system32\cmd.exe

(myenv) D:\mydjango_project\myenv\Scripts>echo %path
%path

(myenv) D:\mydjango_project\myenv\Scripts>echo %path%
D:\mydjango_project\myenv\Scripts;C:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Window
s;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x8
6)\NVIDIA Corporation\PhysX\Common;C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR;C:\Program Files\Git\cmd;C:\Progra
m Files\Java\jdk1.8.0_191\bin;C:\Program Files\MySQL\MySQL Server 8.0\bin;D:\anaconda3\condabin;C:\Users\Dell\AppData\Lo
cal\Programs\Python\Python310\Scripts;C:\Users\Dell\AppData\Local\Programs\Python\Python310;G:\tool\rocketMQ\rocketmq-al
l-4.4.0-bin-release\bin;D:\Program Files (x86)\Tencent\微信web开发者工具\dll;C:\Users\Dell\AppData\Roaming\nvm;C:\Progra
m Files\nodejs;G:\tool\maven\apache-maven-3.8.1\bin;C:\Program Files\nodejs;C:\Users\Dell\AppData\Local\Programs\Python\
Python310\Scripts\;C:\Users\Dell\AppData\Local\Programs\Python\Python310\;C:\Users\Dell\AppData\Local\Microsoft\WindowsA
pps;D:\Program Files (x86)\开发工具\Microsoft VS Code\bin;G:\tool\aria2\aria2-1.37.0-win-64bit-build1;C:\Users\Dell\App
ata\Roaming\npm;C:\Users\Dell\AppData\Roaming\nvm;C:\Program Files\nodejs

(myenv) D:\mydjango_project\myenv\Scripts>
```

```
C:\Windows\system32\cmd.exe

icrosoft Windows [版本 10.0.19045.3208]
(c) Microsoft Corporation。保留所有权利。

:\Users\Dell>echo %path
path

:\Users\Dell>echo %path%
:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Wind
ws\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR;C:\Program Files\Git\cmd;C:\Program Files\Java\jdk1.8.0_191\bin;C:\P
rogram Files\MySQL\MySQL Server 8.0\bin;D:\anaconda3\condabin;C:\Users\Dell\AppData\Local\Programs\Python\Python310\Scri
pts;C:\Users\Dell\AppData\Local\Programs\Python\Python310;G:\tool\rocketMQ\rocketmq-all-4.4.0-bin-release\bin;D:\Program
Files (x86)\Tencent\微信web开发者工具\dll;C:\Users\Dell\AppData\Roaming\nvm;C:\Program Files\nodejs;G:\tool\maven\apach
e-maven-3.8.1\bin;C:\Program Files\nodejs;C:\Users\Dell\AppData\Local\Programs\Python\Python310\Scripts\;C:\Users\Dell\A
ppData\Local\Programs\Python\Python310\;C:\Users\Dell\AppData\Local\Microsoft\WindowsApps;D:\Program Files (x86)\开发工
具\Microsoft VS Code\bin;G:\tool\aria2\aria2-1.37.0-win-64bit-build1;C:\Users\Dell\AppData\Roaming\npm;C:\Users\Dell\App
Data\Roaming\nvm;C:\Program Files\nodejs

:\Users\Dell>
```

唯一的区别就是虚拟环境插入了这行代码 意思激活脚本的作用 相当于截胡了嘛 path环境变量谁在前面就优先用谁

```
C:\Windows\system32\cmd.exe

(myenv) D:\mydjango_project\myenv\Scripts>echo %path
%path

(myenv) D:\mydjango_project\myenv\Scripts>echo %path%
D:\mydjango_project\myenv\Scripts;C:\Program Files (x86)\Common Files\Oracle\Java\javapath;C:\Windows\system32;C:\Window
s;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C:\Program Files (x8
6)\NVIDIA Corporation\PhysX\Common;C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR;C:\Program Files\Git\cmd;C:\Progra
m Files\Java\jdk1.8.0_191\bin;C:\Program Files\MySQL\MySQL Server 8.0\bin;D:\anaconda3\condabin;C:\Users\Dell\AppData\Lo
cal\Programs\Python\Python310\Scripts;C:\Users\Dell\AppData\Local\Programs\Python\Python310;G:\tool\rocketMQ\rocketmq-al
l-4.4.0-bin-release\bin;D:\Program Files (x86)\Tencent\微信web开发者工具\dll;C:\Users\Dell\AppData\Roaming\nvm;C:\Progra
m Files\nodejs;G:\tool\maven\apache-maven-3.8.1\bin;C:\Program Files\nodejs;C:\Users\Dell\AppData\Local\Programs\Python\
Python310\Scripts\;C:\Users\Dell\AppData\Local\Programs\Python\Python310\;C:\Users\Dell\AppData\Local\Microsoft\WindowsA
pps;D:\Program Files (x86)\开发工具\Microsoft VS Code\bin;G:\tool\aria2\aria2-1.37.0-win-64bit-build1;C:\Users\Dell\AppD
ata\Roaming\npm;C:\Users\Dell\AppData\Roaming\nvm;C:\Program Files\nodejs

(myenv) D:\mydjango_project\myenv\Scripts>
```

五、Django基础

5.1 概述

`Django`是一款基于`Python`语言的、开源的、重量级的`WEB`开发框架。

`Django`适用于开发各种类型的`Web`应用程序，包括但不限于电商网站、社交网络、博客、论坛、新闻网站等。

常用的大部分组件:

1.基本配置

2.路由系统

3.原生html模版系统

4.视图view

5.Model模型,数据库链接和ORM数据库管理

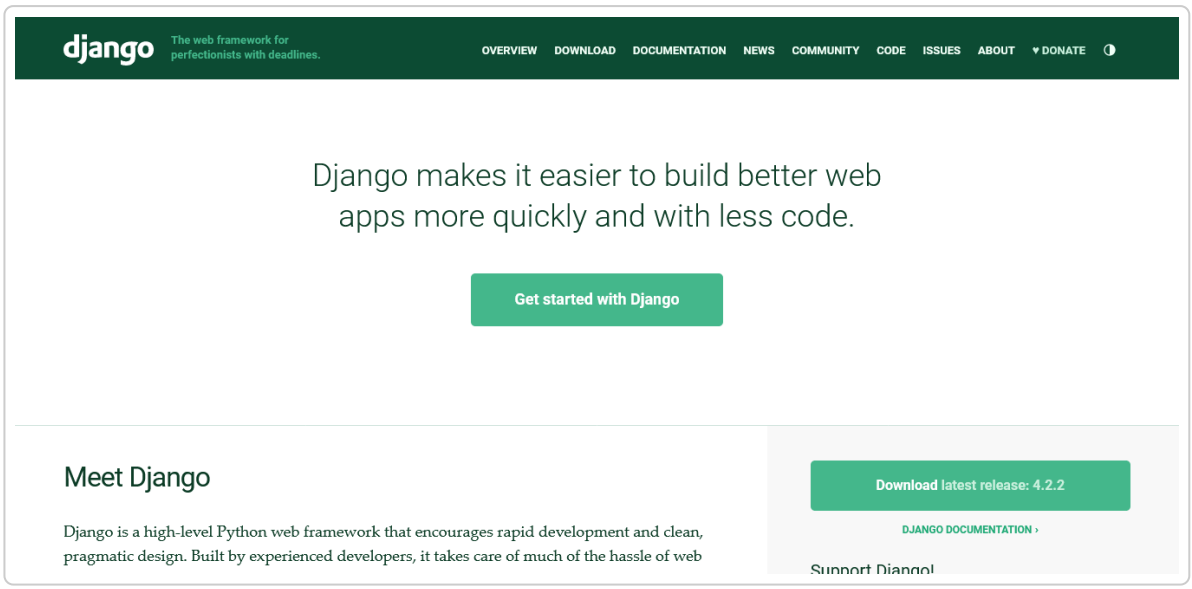
6.中间件

7.Cookie&Session

8.数据后台管理系统admin

`Django` 的官网地址 https://www.djangoproject.com/

中文文档:https://docs.djangoproject.com/zh-hans/3.2/



5.2 安装与卸载

5.2.1 版本要求

`Django` 版本	`Python` 版本
`3.2`	`3.6`、`3.7`、`3.8`、`3.9`、`3.10` (在 `3.2.9` 添加)
`4.0`	`3.8`、`3.9`、`3.10`
`4.1`	`3.8`、`3.9`、`3.10`、`3.11` (在`4.1.3`添加)
`4.2`	`3.8`、`3.9`、`3.10`、`3.11`
`5.0`	`3.9`、`3.10`、`3.11`、`3.12`

5.2.2 安装

在线安装指定版本

```
pip install Django==version
```

如：

```
pip install Django==3.2.20
```

5.2.3 验证

方式1：

```
python -m django --version
```

方式2：

```
django-admin --version
```

方式3：

```
>>> import django

>>> django.get_version()
```

> 可通过`pip3 show <package>`命令来显示已安装的软件包的信息，如：
>
> ```
> pip show django
> ```

5.2.4 卸载

```
pip uninstall django
```

六、`Django`项目

6.1 概述

`Django`的项目通常由一个或多个应用程序（也称为模块）组成，每个应用程序都包含了与特定功能相关的模型、视图、模板和静态文件。

`Django`提供了一些内置应用程序，如认证、管理、会话等，同时也支持第三方应用程序的集成。这些应用程序可以通过`Django`的包管理器`pip`进行安装。

6.2 创建

在`Django` 中创建项目需以通过以下三个步骤实现：

第一步：创建`Django`项目

```
python -m django startproject djangoBasic
```

其中：

- `djangoBasic`是项目名称，由用户自行指定合法的名称（避免使用 `Python` 或 `Django` 的关键字）

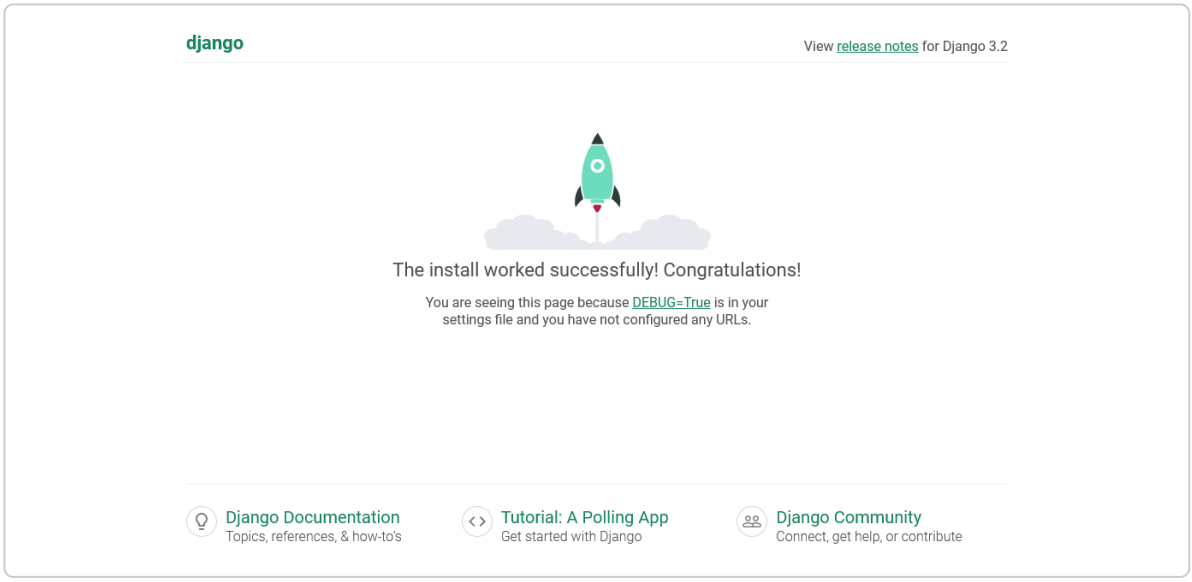
第二步：启动`Django`项目

```
# 切换到djangoBasic目录
cd djangoBasic

# 运行项目
python manage.py runserver
```

第三步：访问`Django`项目

启动浏览器，然后在地址栏输入`http://127.0.0.1:8000`，浏览效果如下图所示：



6.3 目录结构

创建`Django`项目后，其目录结构如下：

```
djangoBasic/
├── djangoBasic
|   ├── asgi.py
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   └── wsgi.py
└── manage.py
```

这些目录和文件的用处是：

- 最外层的`djangoBasic`目录是项目的根目录，该目录也是项目的容器
- `manage.py`是一个管理`Django`项目的命令行工具
- 内层的`djangoBasic`目录是一个`Python`包，它的名称就是当引用它内部任何东西时所需要用到的`Python`包名
- `djangoBasic/__init__.py`，一个空文件，其作用是告诉`Python`解释器：该目录是一个`Python`包
- `djangoBasic/settings.py`，`Django`项目的配置文件
- `djangoBasic/urls.py`，`URL`调度器（也称为路由配置文件）
- `djangoBasic/asgi.py`，运行在`ASGI`兼容的`Web`服务器上的入口
- `djangoBasic/wsgi.py`，运行在`WSGI`兼容的`Web`服务器上的入口

6.4 配置文件

在`Django`的项目中，其配置文件位置于项目的根目录下，名称为`settings.py`，其基本结构如下：

```python
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.

# 当前项目的绝对路径
BASE_DIR = Path(__file__).resolve().parent.parent

# 密钥,用于提供加密签名，并且应该设置为一个唯一的、不可预测的值
SECRET_KEY = 'django-insecure-u^slk-r1pmaa1!+ad_l$j9^m&gui@esm1-9m08dy(%wukz(-
%_'

# 是否开启调试模式，建议：在开发环境开启调试模式，在生产环境关闭调试模式
DEBUG = True

# 当前项目可以服务的主机 / 域名的字符串列表
ALLOWED_HOSTS = []

# 当前项目已安装且被启用的应用程序列表
INSTALLED_APPS = [ ... ]

# 当前项目已安装且被启用的中间件列表
MIDDLEWARE = [ ... ]
```

```
# 根URL调度器的完整导入路径
ROOT_URLCONF = 'djangoBasic.urls'

# 模板引擎的配置的列表
TEMPLATES = [ ... ]

WSGI_APPLICATION = 'djangoBasic.wsgi.application'

# 数据库配置
DATABASES = { ... }

# 密码强度的验证器列表
AUTH_PASSWORD_VALIDATORS = [ ... ]

# 语言代码
LANGUAGE_CODE = 'en-us'

# 时区的字符串
TIME_ZONE = 'UTC'
# 数据本地格式化
USE_I18N = True
#数据本地化
USE_L10N = True
#是否使用感知时间
USE_TZ = True

# 静态文件路由
STATIC_URL = '/static/'

# 默认主键字段的类型
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

请修改以下配置后刷新浏览器：

```
# 语言代码
LANGUAGE_CODE = 'zh-Hans'

# 时区的字符串
TIME_ZONE = 'Asia/Shanghai'
```

七、三、URL调度器

7.1 概述

`Django` 允许程序员自由地设计 `URL`，不受框架束缚。

为了给`Django`项目设计 `URL`，需要创建 `Python` 模块，其通常被称为 `URLconf` （`URL configuration`，`URL`配置）。这个模块是纯粹的 `Python` 代码，包含 `URL` 模式（简单的正则表达式）到 `Python` 函数（视图）的简单映射。

`URL` 调度器也被称为`URL` 路由或者`URLConf`或者`URL` 路由分发器。

`URL`调度器被定义在`urls.py`文件中，其结构如下：

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

7.2 URL路由操作三部曲

第一步：定义`URL`路由

示例代码如下：

```
path('hello/',views.hello),
```

第二步：定义视图函数

示例代码如下：

```
from django.http import HttpResponse

def hello(request):
    return HttpResponse('Hello,Django')
```

第三步：启动浏览器，然后进行访问

如 `http://127.0.0.1:8000/hello/`，运行效果如下图所示：

7.3 `path()` 函数

`path()` 函数的作用是返回一个 `URLPattern` 对象，以便包含在 `urlpatterns` 中

该函数位置于 `django.urls` 模块内，其语法是：

```
from django.urls import path

path(route, view, kwargs=None, name=None)
```

其中：

- `route` 参数是一个匹配 `URL` 准则的字符串

在响应一个请求时，它会从 `urlpatterns` 的第一项开始，按顺序依次匹配列表中的项，直到找到匹配的项为止

- `view` 参数是视图函数名称，该函数用于处理用户的请求并且返回响应
- `kwargs` 参数允许向视图函数传递附加参数，它必须是字典
- `name` 参数为一个 `URL` 路由命名并能在 `Django` 的任意地方唯一地引用它

> 默认情况下，`Django` 会自动为没有斜线结尾的路径添加斜线，并重定向到带有斜线的路径。
>
> 这是通过 `APPEND_SLASH` 设置来实现的，该设置默认为 `True`。
>
> 如果定义了路由规则为 `path('home/', home)`，那么当用户访问 `/home` 时，`Django` 会自动将其重定向到 `/home/`

7.4 视图函数

视图函数（简称为视图）是一个 `Python` 函数，负责处理用户的请求并返回响应

每个视图函数都将 `HttpRequest` 对象作为第一个参数，通常命名为 `request`，同时视图函数都要返回 `HttpResonse` 对象，其语法结构是：

```
from django.http import HttpResponse

def view_function_name(request[,argument1[,argument2[,...]]]):
    ...
    ...
    return HttpResponse(...)
```

其中：

- `request`参数代表当前`HTTP`请求，其必须为第一个参数
- `argument1`,`argument2[,...]`等是可选参数，它们代表从`URL`中传递进来的参数。

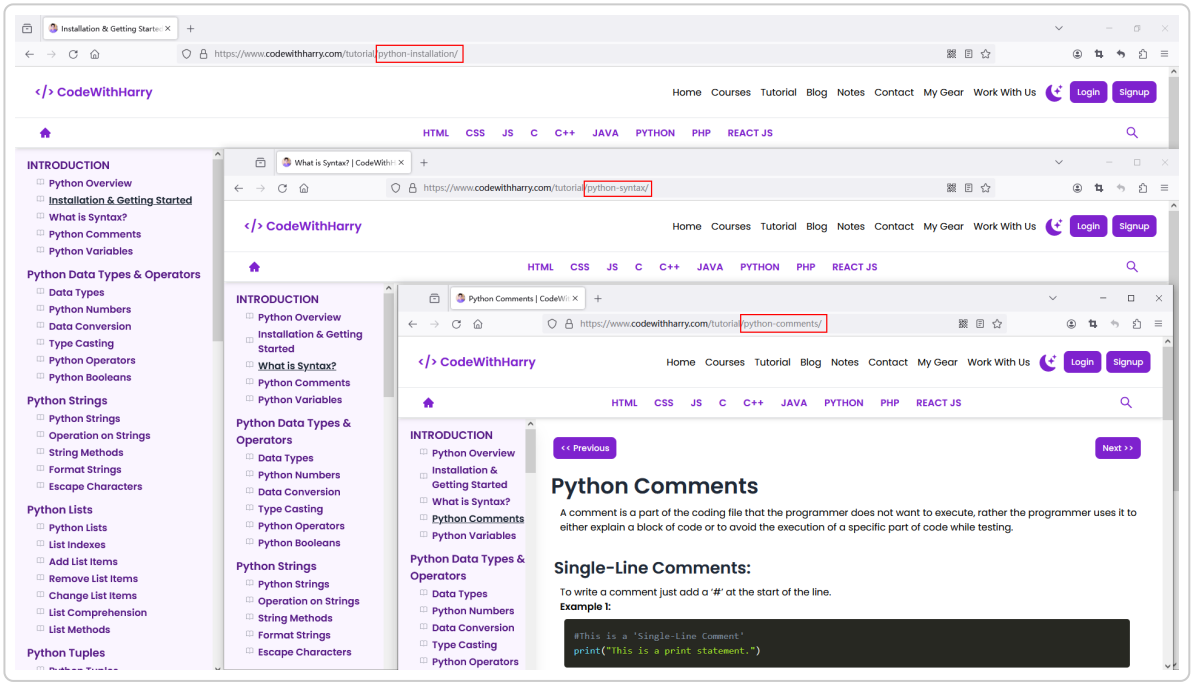
视图函数可以根据需要接收这些参数，并使用它们来生成`HTTP`响应

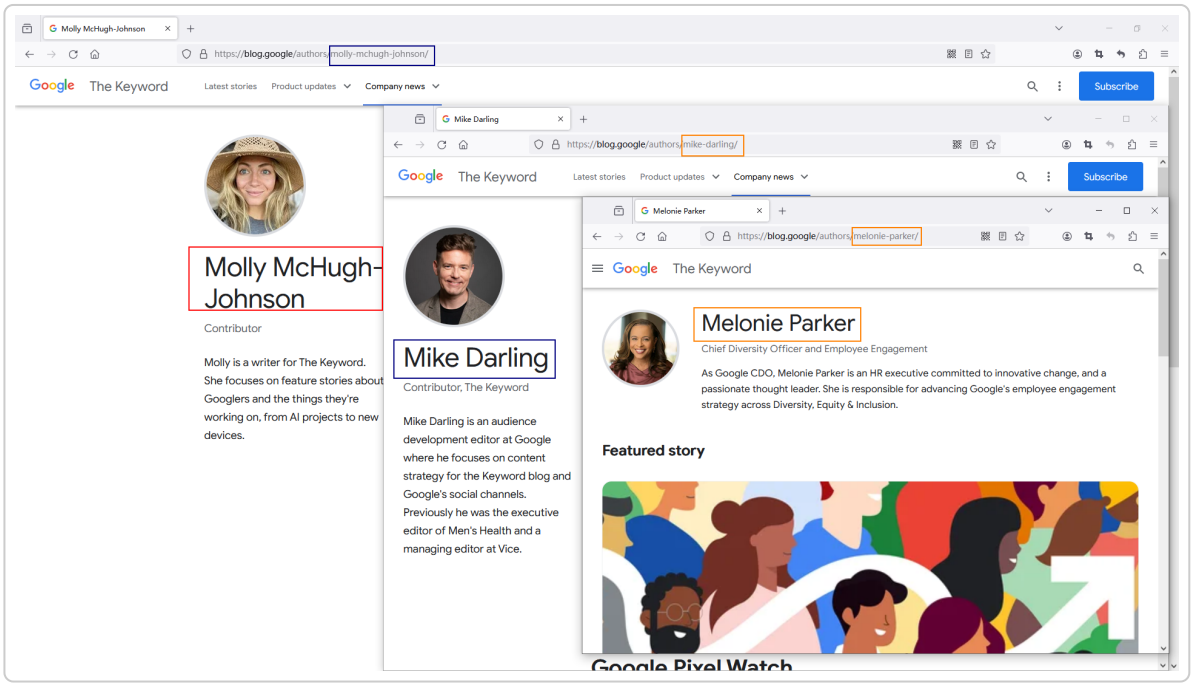
- `HttpResponse`对象用来构造响应对象。`HttpResonse`类位于`django.http`模块中

> 视图函数本质就是自定义函数，之所以称为视图函数的原因：
>
> `Django`采用了`MTV`（`Model`-`Template`-`View`）架构，其中视图函数属于`MVC`架构中的`View`层，故称为视图函数。
>
> 在`Django`中，视图函数的作用就是接收请求，处理请求并返回响应。

7.5 (path)路径转换器

针对如下图所示的`URL`中存在动态信息，如用户`ID`，在`Django`中提供了路径转换器以便在视图函数中捕获和使用这些数据。





路径转换器提供了多种类型的转换器，例如字符串、整数、`slug`、`UUID` 等，以适应不同类型的数据。

使用路径转换器可以轻松地编写灵活且可重用的 `URL` 模式。在 `Django` 中路径转换器包含以下5种：

- `<int:parameter>`：匹配 0 或者任何正整数，返回 `int`
- `<str:parameter>`：匹配除了 `/` 之外的任何非空字符串，不包括特殊字符或符号，直到下一个斜杠为止
- `<slug:parameter>`：匹配任何 `ASCII` 字母、数字字符、连字符(`-`)和下划线(`_`)组成的字符
- `<uuid:parameter>`：匹配 `UUID` 格式的字符串；其格式是一个由 `32` 个十六进制字符组成的字符串
- `<path:parameter>`：匹配非空字符串，包括路径分隔符(`/`)。通常用于捕获整个 `URL` 路径

路径转换器中的参数与视图函数参数之间的对应关系如下图所示：

- 整数转换器适用于需要将 `URL` 中的数字部分作为整数处理的场景，如博客文章的 `ID` 或页码等
- 字符串转换器适用于需要将 `URL` 中的字符串部分作为普通字符串处理的场景，如用户名或标签等
- `Slug` 转换器适用于需要将 `URL` 中的字符串部分作为 `slug` 格式处理的场景，如文章标题或 `URL` 友好的名称
- `UUID` 转换器适用于需要将 `URL` 中的字符串部分作为 `UUID` 对象处理的场景，如唯一标识符或密钥等

7.6 实践建议

为了便于代码的维护，强烈建议将视图函数统一定义在名为 `views.py` 的文件里，这个文件放置在项目或应用目录里。此时的目录结构如下：

```
djangoBasic/
├── djangoBasic
|   ├── asgi.py
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── views.py
|   └── wsgi.py
└── manage.py
```

7.7 `re_path()` 函数

`re_path()` 函数用于定义 `URL` 路由规则，并将不同的 `URL` 请求映射到对应的视图函数上。它可以使用正则表达式来匹配 `URL`，从而实现更加灵活的路由控制，其语法结构是：

```
re_path(route, view, kwargs=None, name=None)
```

> `re_path()` 函数本身不支持使用路径转换器，但是可以在正则表达式中使用分组语法来实现类似的功能

> `re_path()` 函数返回 `URLPattern` 对象

其中：

- `route` 参数是一个正则表达式字符串，用于匹配 `URL`。另外，可用 `(?P<name>pattern)` 命名捕获组，以便视图函数可以从 `URL` 中获取参数信息，其中 `name` 是组名，`pattern` 是要匹配的模式
- `view` 参数是一个处理请求的视图函数，该函数用于处理用户的请求并且返回响应
- `kwargs` 参数允许向视图函数传递附加参数，它必须是字典
- `name` 参数为一个 `URL` 路由命名并能在 `Django` 的任意地方唯一地引用它

7.8 `URL` 调度器的工作流程

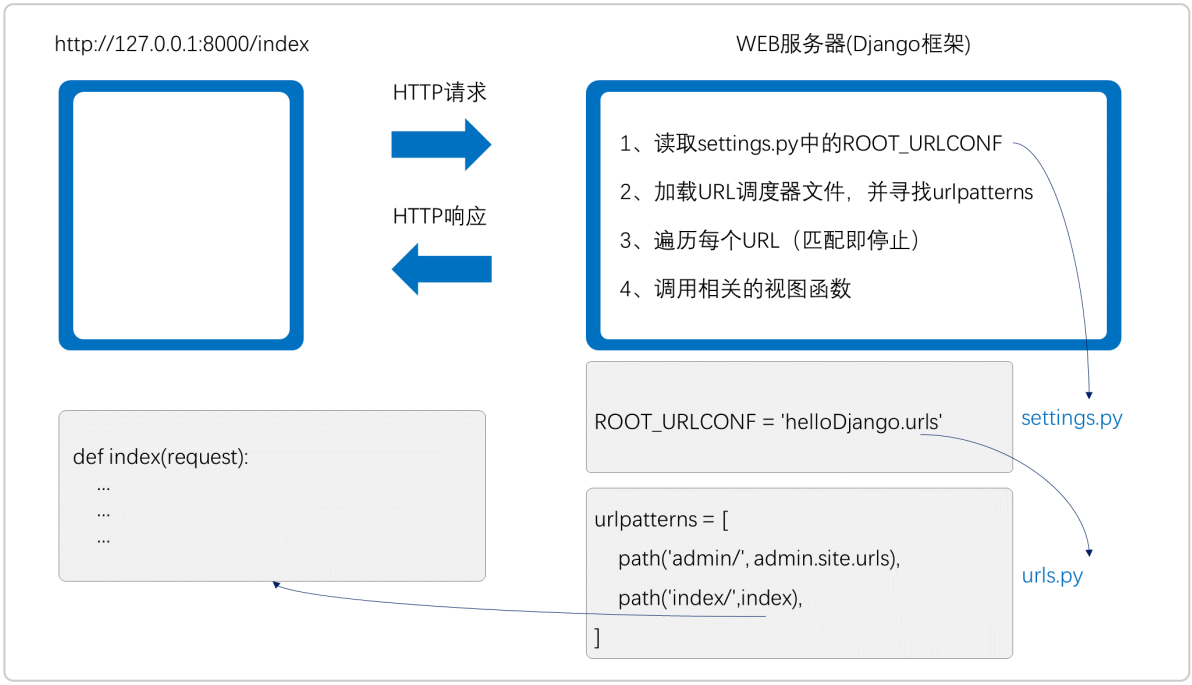
第一步：`Django` 确定使用根 `URLconf` 模块。通常是 `ROOT_URLCONF` 设置的值；

第二步：`Django` 加载该 `Python` 模块并寻找可用的 `urlpatterns`；

第三步：`Django` 会按顺序遍历每个 `URL` 模式，然后会在所请求的 `URL` 匹配到第一个模式后停止；

第四步：一旦有 `URL` 匹配成功，`Djagno` 导入并调用相关的视图函数；

第五步：如果没有 `URL` 被匹配，或者匹配过程中出现了异常，`Django` 会调用一个适当的错误处理视图。



7.9 `URL` 路由操作三部曲

第一步：`urls.py` 中定义路由，并且指定视图函数

第二步：`views.py` 中定义视图函数并返回 `HttpResponse` 响应

第三步：在浏览器直接输入地址或者在视图函数中响应 `HTML` 链接(`a` 标签)到客户端，然后进行访问

八、应用

8.1 概述

应用`(Application，简称APP)`，在`Django`中，每个项目的的独立业务模块就是一个应用。它包括模型，视图，模板，模板标签，静态文件等的组合。

8.2 创建应用

在`Django`中创建应用可以通过下列两个步骤实现：

第一步：创建应用

```
python manage.py startapp name
```

示例如下：

```
python manage.py startapp app01
```

第二步：注册应用

在`settings.py`的`INSTALLED_APPS`列表中添加已创建的应用，例如：

```
#注册应用
INSTALLED_APPS = [
    ...
    'app01',
    ...
]
```

8.3 应用的目录结构

```
jangoBasic/
├── djangoBasic
|   ├── ...
├── manage.py
└── users
    ├── admin.py
    ├── apps.py
    ├── __init__.py
    ├── migrations
    |   └── __init__.py
    ├── models.py
    ├── tests.py
    └── views.py
```

这些文件的用处是：

- `users/admin.py` 文件，后台管理配置文件
- `users/apps.py` 文件，配置文件
- `users/__init__.py` 文件，初始化文件
- `users/migrations` 目录，数据迁移目录
- `usrs/models.py` 文件，模型类文件
- `tests.py` 文件，单元测试文件
- `views.py` 文件，视图函数处理文件

8.4 `include()` 函数

`include()` 函数包含在 `django.urls` 模块中，其作用是引用其它 `URLconfs`。每当 `Django` 遇到 `include()` 时，它会截断与此项匹配的 `URL` 的部分，并将剩余的字符串发送到 `URLconf` 以供进一步处理。语法结构是：

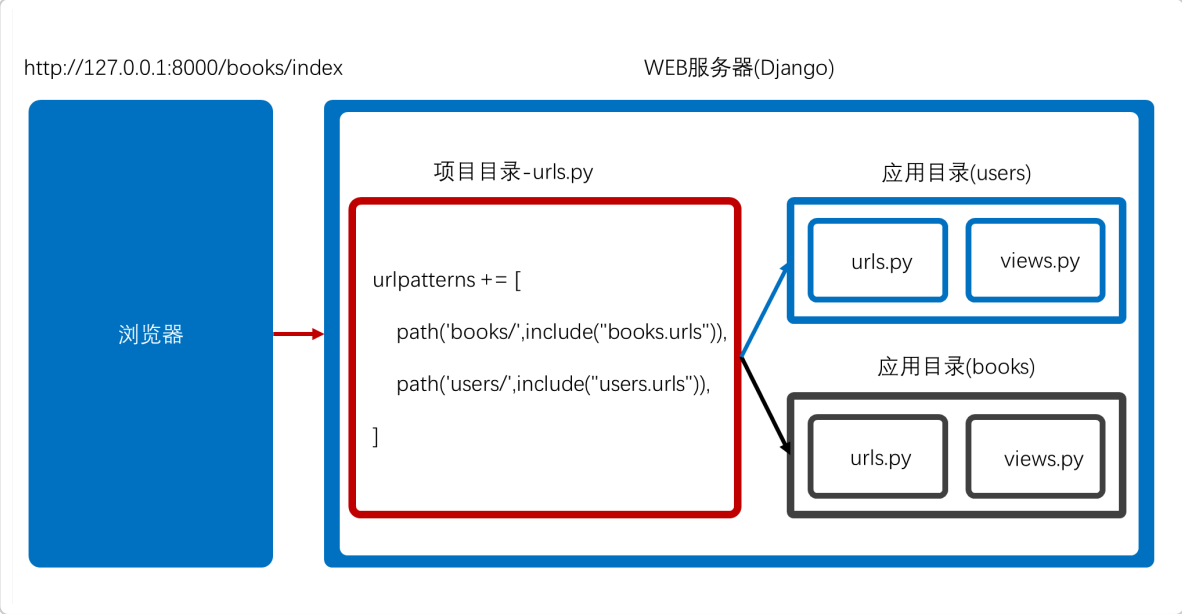
```
from django.urls import include

include(module, namespace=None)
```

其中：

- `module` -- `URLconf` 模块（或模块名称）
- `namespace` -- 包含的 `URL` 条目的实例命名空间

8.5 分布式 `url` 调度器



九、模板

9.1 概述

`Django`中的模板是指可以动态生成任何基于文本格式文件的技术（如`HTML`、`XML`、`CSV`、`JS`、`CSS`等）。

> `MVC`
>
> `Model`模型，主要用于数据的读写操作
>
> `Controller`控制器，主要用于接收用户的请求并且返回响应到客户端
>
> `View`视图，主要用于显示的`HTML`页面及相关技术(如`CSS`、`JavaScript`)
>
> `MVT`
>
> `Model`模型，主要用于数据的读写操作
>
> `View`视图，主要用于接收用户的请求并且返回响应到客户端
>
> `Template`模板，主要用于显示的`HTML`页面及相关技术(如`CSS`、`JavaScript`)

`Django`内置了自己的模板系统，称为`Django`模板语言(`DTL`，`Django Template Language`)。

由于历史原因，模板引擎的通用支持和`Django`模板语言的实现都在`django.template`的命名空间中。

在`Django 3.2`中，支持两种模板引擎：默认的`Django`模板引擎和第三方的`Jinja2`模板引擎。

9.2 配置

`settings.py`中关于模板的配置如下：

```
TEMPLATES = [
    {
        # 使用的模板后端
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        # 模板引擎应该查找模板源文件的目录,将按照顺序依次搜索（原则：查到即停止）
        'DIRS': [BASE_DIR / 'templates'],
        # 模板引擎是否应该在已安装的应用程序的模板目录中查找模板源文件
        'APP_DIRS': True,
        # 模板引擎的选项
        'OPTIONS': {
            # ... some options here ...
        },
    },
]
```

在`djangoTemplates`项目的根目录下创建名称为`templates`的目录，以便存储模板文件：

```
$ cd djangoTemplates
$ mkdir djangoTemplates/templates
```

此时的目录结构如下：

```
djangoTemplates/
├── djangoTemplates
|   ├── asgi.py
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── views.py
|   └── wsgi.py
├── manage.py
└── templates
```

十、便捷视图函数

在 `django.shortcuts` 包收集助手函数，可以协助程序员快速完成某些工作。

10.1 `render()` 函数

`render()` 函数用于将给定的模板与给定的上下文字典组合在一起，并以渲染的文本返回 `HttpResponse` 对象。

其语法结构是：

```
from django.shortcuts import render

render(request, template_name, context=None, content_type=None, status=None, using=None)
```

> `render()` 函数返回一个 `HttpResponse` 对象

其中：

- `request` 参数指用于生成此响应的请求对象
- `template_name` 参数用于指定要使用的模板的名称
- `context` 参数用于指定要添加到模板上下文的值所组成的字典
- `context_type` 参数用来指定模板文件的 `MIME` 类型，默认 `'text/html'`
- `status` 参数用来指定 `HTTP` 响应的状态码，默认为 `200`
- `using` 参数用于指定加载模板的模板引擎的 `NAME`

示例代码如下：

```
from django.shortcuts import render

def index(request):
    return render(request,'index.html')
```

10.2 `redirect()` 函数

`redirect()` 函数用于 **重定向到指定的 URL 或视图**，返回一个 `HttpResponseRedirect` 对象（HTTP 302 临时重定向）或 `HttpResponsePermanentRedirect` 对象（HTTP 301 永久重定向）。常用于处理表单提交后跳转、旧 URL 迁移等场景。

其语法结构是：

```
from django.shortcuts import redirect

redirect(to, *args, permanent=False, **kwargs)
```

> `render()` 函数返回一个 `HttpResponse` 对象

10.3 `render()` 函数与 `HttpResponse` 对象

`render()` 函数和 `HttpResponse` 对象在 `Django` 中都用于处理 `HTTP` 响应，但它们的作用不同：

- `render()` 函数通常用于将模板和上下文数据组合成一个 `HttpResponse` 对象来返回给客户端
- `HttpResponse` 对象则是直接返回一个 `HTTP` 响应，通常用于返回纯文本或者其他非 `HTML` 格式的数据

十一、考试

11.1 本考小考

1. 什么是 `Django`？
2. `Django` 可以做什么？
3. 如何安装 `Django`？
4. 如何定义路由？
5. 路由的工作流程是什么？